

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Департамент программной
инженерии ФКН НИУ ВШЭ
Научный руководитель,
кандидат технических наук,
доцент департамента
программной инженерии
факультета компьютерных наук

_____ / Н.С. Белова /
«__» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.02-06 ТЗ 01–1 ЛУ

Исполнители:
студент группы БПИ233

_____ / Б.А. Багавиев /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН
RU.17701729.02-06 ТЗ 01–1 ЛУ

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Техническое задание

RU.17701729.02-06 ТЗ 01–1

Листов 27

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
1.1. Наименование темы разработки	1
1.2. Наименование программы	1
1.3. Состав комплекса программ	1
1.4. Краткая характеристика области применения	2
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ	3
2.1. Документы, на основании которых ведётся разработка	3
2.2. Наименование темы разработки	3
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ	4
3.1. Функциональное назначение программы	4
3.2. Эксплуатационное назначение программы	5
3.2.1. Архитектура развёртывания	5
3.2.2. Требования к окружению	5
3.2.3. Сценарии использования	5
4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ	6
4.1. Требования к функциональным характеристикам	6
4.1.1. Управление топиками (Management API)	6
4.1.2. Управление кластером (Management API)	6
4.1.3. Публикация сообщений (Data API)	7
4.1.4. Чтение сообщений (Data API)	7
4.1.5. Группы консьюмеров (Data API)	7
4.1.6. Алгоритм диапазонного назначения партиций	8
4.1.7. Аутентификация	8
4.2. Требования к надёжности	8
4.3. Требования к составу и параметрам технических средств	9
4.4. Требования к информационной и программной совместимости	9
4.5. Специальные требования	10
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	11
5.1. Состав программной документации	11
5.2. Специальные требования к документации	11
5.2.1. Требования к оформлению	11
5.2.2. Требования к содержанию пояснительной записки	12
5.2.3. Требования к руководству оператора	12
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	13
6.1. Экономическая эффективность	13
6.1.1. Снижение лицензионных затрат	13
6.1.2. Снижение требований к инфраструктуре	13
6.1.3. Прогнозируемые технические характеристики	13

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	15
7.1. Сроки разработки и исполнители	16
8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ	17
8.1. Критерии приёмки	17
8.2. Процедура приёмки	18
8.3. Требования к результатам приёмки	18
ManagementService	23
DataService	23
Коды ошибок	24

1. ВВЕДЕНИЕ

1.1. Наименование темы разработки

Наименование темы разработки – «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**».

1.2. Наименование программы

Наименование программы – «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**» (далее – «Брокер», «Система», «BunnyMQ»).

Наименование на английском языке – «**Go message queue with delivery guarantees and horizontal scaling**».

Краткое наименование: **BunnyMQ**.

Обозначение программного документа по ГОСТ 19.103-77: **RU.17701729.02-06 ТЗ 01–1** (числовой код вида документа по ГОСТ 19.101-77: **05**).

Вид программного изделия по ГОСТ 19.101-77, §2.2: **комплекс программ** – совокупность взаимодействующих программных компонентов, объединённых в единую систему: серверный процесс брокера, клиентская библиотека и утилита командной строки.

1.3. Состав комплекса программ

Комплекс BunnyMQ включает три компонента:

1. `bunmq` (`cmd/bunmq`) – серверный процесс брокера; запускает gRPC-серверы, узел Raft, координаторы и HTTP-эндпоинт метрик;
2. `bunmq-cli` (`cmd/bunmq-cli`) – утилита командной строки для управления кластером и тестирования: команды `produce`, `consume`, `topic`, `cluster`;
3. `pkg/client` – публичная Go-библиотека с типами `Producer`, `Consumer`, `AdminClient` и автоматическим поиском лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.4. Краткая характеристика области применения

BunnyMQ предназначен для инфраструктуры микросервисных приложений, где требуется надёжная асинхронная передача сообщений с гарантиями доставки и возможностью горизонтального масштабирования.

Типичные сценарии:

- передача событий между сервисами с гарантией «хотя бы однократной» доставки (`acks=all`);
- сбор телеметрии и логов от большого числа продюсеров (`acks=0`);
- организация нескольких независимых потоков данных через механизм топиков и партиций;
- параллельная обработка данными несколькими консьюмерами через группы с автоматическим перераспределением партиций.

Категории пользователей:

- **Разработчики приложений-продюсеров** – подключаются через `pkg/client` или `gRPC` напрямую, отправляют сообщения в топик с выбором уровня подтверждения;
- **Разработчики приложений-консьюмеров** – читают из партиций по смещению, при необходимости используют группы консьюмеров;
- **Администраторы кластера** – управляют топологией через `bunmq-cli` или `Management API`: создают и удаляют топик, меняют настройки хранения, просматривают состояние кластера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1. Документы, на основании которых ведётся разработка

Разработка ведётся на основании следующих документов:

1. Устав федерального государственного автономного образовательного учреждения высшего образования «Национальный исследовательский университет «Высшая школа экономики»» (с изменениями) № 56 от 1.02.2016.
2. Приказ декана ФКН НИУ ВШЭ И. В. Аржанцева № 2.3-02/1112–04 от 11.12.2025 «Об утверждении тем, руководителей курсовых работ студентов образовательной программы Программная инженерия факультета компьютерных наук».

Организация, утвердившая документ-основание разработки: **Факультет компьютерных наук НИУ ВШЭ.**

Дата утверждения документа-основания: **11.12.2025.**

2.2. Наименование темы разработки

Наименование темы разработки — «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение программы

BunnyMQ реализует полный цикл работы с потоками сообщений.

Управление топиками. Создание топика с указанием числа партиций, фактора репликации и параметров хранения; удаление с асинхронной физической очисткой данных; получение списка топиков и метаданных конкретного топика (лидеры партиций, расположение реплик); изменение числа партиций (только увеличение) и параметров хранения (`retention_ms`, `retention_bytes`).

Управление кластером. Получение состояния кластера: список узлов с адресами и статусом; список партиций топика с лидером и репликами. Состав кластера задаётся статически при запуске.

Публикация сообщений. Приём пакета сообщений и запись в партицию. Режим `acks=0` – без ожидания подтверждения, через `dragonboat.Propose`. Режим `acks=all` – с ожиданием кворума реплик, через `dragonboat.SyncPropose`; продюсер получает назначенное смещение. Режим `acks=1` намеренно не реализован.

Чтение сообщений. Получение записей начиная с заданного смещения и до `max_bytes` байт. При отсутствии новых данных запрос удерживается до `max_wait_ms` миллисекунд (long-poll). Работает только на узле-лидере партиции.

Запросы смещений. Наименьшее (`earliest`) и наибольшее (`latest`) смещения партиции; поиск первого смещения с временной меткой не ранее заданной.

Группы консьюмеров. Вступление в группу с назначением `member_id` и `generation_id`; периодический `heartbeat`; явный выход; фиксация смещений и чтение зафиксированных смещений. При изменении состава группы запускается ребалансировка с диапазонным назначением партиций

Аутентификация. Токен в gRPC-метаданных (`bunnymq-auth-token`), проверяется по списку, заданному при запуске. Пустой список – режим PLAINTEXT.

Мониторинг. Экспорт метрик в формате Prometheus на каждом узле.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2. Эксплуатационное назначение программы

3.2.1. Архитектура развёртывания

BunnyMQ запускается как кластер из нечётного числа узлов (как правило, 3 или 5), где каждый узел – один процесс `bunnymq` с выделенным дисковым разделом для хранения журналов партиций. Внутри кластера работает `multi-raft` через `dragonboat v4`: один `shard` на партицию и один `shard` для метаданных. Клиент, попавший на нелидера, получает `NotLeader` с адресом лидера и переключается автоматически.

3.2.2. Требования к окружению

- Операционная система: Linux (ядро 5.4+) или macOS 12+, платформа amd64 или arm64.
- Версия Go: 1.25 и выше (для сборки из исходников).
- Дисковое хранилище: рекомендуется SSD; файловая система – ext4, xfs или apfs.
- Сеть: TCP-связность между всеми узлами на портах Raft и gRPC.

3.2.3. Сценарии использования

Продюсер создаёт подключение через `pkg/client.NewProducer`. Библиотека сама находит лидера нужной партиции. При `acks=all` получает подтверждённое смещение; при `acks=0` ответ возвращается немедленно.

Консьюмер без группы вызывает `Fetch` напрямую, задавая топик, партицию и смещение. Фиксация смещений опциональна.

Консьюмер с группой вызывает `Subscribe`. Библиотека выполняет `JoinGroup`, получает назначенные партиции и держит в фоне горутину `heartbeat`. При сигнале ребалансировки повторяет `JoinGroup`.

Администратор управляет топиками и смотрит состояние кластера через `bunnymq-cli` или `AdminClient`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ

4.1. Требования к функциональным характеристикам

4.1.1. Управление топиками (Management API)

Брокер должен реализовывать операции управления топиками через gRPC-сервис `ManagementService`:

1. **Создание топика** (`CreateTopic`): входные параметры – `name` (уникальное имя, `[a-zA-Z0-9._-]{1,255}`), `partition_count` (≥ 1), `replication_factor` ($\geq 1, \leq$ числа узлов), опциональные `retention_ms`, `retention_bytes`. При существующем топике – `TopicAlreadyExists`.
2. **Удаление топика** (`DeleteTopic`): физическое удаление данных асинхронно; при отсутствии – `TopicNotFound`.
3. **Список топиков** (`ListTopics`): возвращает все топика с полями `name`, `partition_count`, `replication_factor`.
4. **Описание топика** (`DescribeTopic`): для каждой партиции – `id` лидера и список `id` реплик с их статусом.
5. **Изменение числа партиций** (`AlterTopicPartitions`): только увеличение; уменьшение не поддерживается.
6. **Изменение политики хранения** (`AlterTopicRetention`): обновляет `retention_ms` и / или `retention_bytes`; применяется на следующем цикле очистки.

4.1.2. Управление кластером (Management API)

1. **Описание кластера** (`DescribeCluster`): список узлов с полями `node_id`, `address`, `status`.
2. **Список партиций топика** (`ListPartitions`): для каждой партиции – `partition_id`, `leader_node_id`, `replica_node_ids`, диапазон смещений.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. Состав кластера задаётся статически в конфигурационном файле; добавление и удаление узлов в рантайме в первой версии не поддерживается.

4.1.3. Публикация сообщений (Data API)

1. **Produce:** входные параметры – `topic`, `partition_id` (если не задан – по хешу ключа FNV-1a или round-robin), `batch_data`, `acks` (0 или all). При `acks=all` возвращает смещение первого сообщения пакета; при `acks=0` – ответ немедленно, без смещения.
2. Все сообщения одного пакета записываются в одну партицию с последовательными смещениями атомарно (всё или ничего).
3. Формат `batch_data` на диске совпадает с форматом на проводе; клиент кодирует пакет, сервер хранит без изменений.

4.1.4. Чтение сообщений (Data API)

1. **Fetch:** входные параметры – `topic`, `partition_id`, `offset`, `max_bytes`, `max_wait_ms`. Возвращает записи начиная с `offset` до исчерпания `max_bytes`. Если записей нет – удерживает запрос до появления данных или истечения `max_wait_ms`.
2. **GetOffsets:** возвращает `earliest_offset` и `latest_offset` для партиции; поддерживается поиск смещения по временной метке (`GetOffsetByTimestamp`).
3. `Fetch` и `GetOffsets` обслуживаются только на узле-лидере; при обращении к не-лидеру – `NotLeader` с адресом лидера.

4.1.5. Группы консьюмеров (Data API)

1. **JoinGroup:** консьюмер указывает `group_id` и список топиков. Сервер добавляет участника, выполняет диапазонное назначение, возвращает `member_id`, `generation_id` и список партиций.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Heartbeat**: консьюмер периодически (рекомендуется раз в 3 секунды) отправляет heartbeat с `group_id`, `member_id`, `generation_id`. При устаревшем `generation_id` ответ содержит `REBALANCE_REQUIRED` – консьюмер должен повторить `JoinGroup`.
3. **LeaveGroup**: явный выход из группы; запускает немедленную ребалансировку с новым `generation_id`.
4. **CommitOffset**: фиксация смещений по одной или нескольким партициям. Требуется актуальных `member_id` и `generation_id`.
5. **FetchCommittedOffsets**: возвращает последние зафиксированные смещения для пар (`group_id`, `partition`).
6. Если консьюмер не отправлял heartbeat дольше `session timeout` (по умолчанию 30 секунд), координатор исключает его из группы и инициирует ребалансировку

4.1.6. Алгоритм диапазонного назначения партиций

Для каждого топика: партиции сортируются по `partition_id`, участники группы – по `member_id`. Диапазон партиций делится на `len(members)` частей; первые $P \bmod M$ участников получают на одну партицию больше. Алгоритм идентичен `Range Assignment` в `Apache Kafka`.

4.1.7. Аутентификация

Токен передаётся в заголовке gRPC-метаданных `bunnumq-auth-token`. Сервер проверяет по списку из конфигурации. Пустой список – аутентификация отключена (режим `PLAINTEXT`). Недействительный токен – `Unauthenticated`.

4.2. Требования к надёжности

1. При отказе одного узла из трёх кластер продолжает обслуживать запросы.
2. Данные, подтверждённые с `acks=all`, не теряются при отказе одного узла.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. Узел после перезапуска восстанавливает Metadata FSM из снимка и воспроизведения команд Raft; Partition FSM восстанавливает Storage из журнала на диске.
4. Storage корректно обрабатывает torn write: при открытии активного сегмента хвост усекается до последней полной записи.
5. Очистка сегментов по политике хранения (DeleteSegmentsBefore) реализована как команда Raft – все реплики удаляют одни и те же данные.
6. Graceful shutdown: узел прекращает приём новых RPC, дожидается завершения текущих, синхронизирует Storage и корректно завершает dragonboat.

4.3. Требования к составу и параметрам технических средств

Таблица 4.1 — Минимальные требования к аппаратным средствам

Параметр	Минимальное значение
CPU	2 ядра x86-64 на узел
Оперативная память	512 МБ на узел (без учёта данных)
Дисковое хранилище	SSD или HDD; объём – retention_bytes × число партиций
Операционная система	Linux (ядро 5.4+) или macOS 12+
Сеть	TCP-связность между всеми узлами; задержка < 100 мс

4.4. Требования к информационной и программной совместимости

1. Язык реализации: Go 1.25 и выше.
2. Транспорт: gRPC (HTTP/2), Protobuf. Совместимость с любым gRPC-клиентом, использующим сгенерированные из api/*.proto stub-файлы.
3. Raft: библиотека dragonboat v4 (github.com/ltni/dragonboat/v4); взаимодействие между узлами только через dragonboat.
4. Метрики: формат Prometheus text/plain, совместимый с Prometheus Server 2.x и OpenMetrics.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. Формат журнала на диске совместим с форматом пакетов Apache Kafka, что допускает замену транспорта без изменения слоя хранения.
6. Внешних зависимостей на базы данных, очереди или объектные хранилища нет; все данные хранятся локально.

4.5. Специальные требования

1. Partition FSM (IOnDiskStateMachine) строго детерминирован: метод `Update()` не вызывает `time.Now()` и не производит сетевых операций
2. Логирование: структурированный JSON через `go.uber.org/zap`; уровни `debug`, `info`, `warn`, `error`.
3. Конкурентность управляется через `goroutine` и `channel`; глобального состояния между запросами нет.
4. Метрики экспортируются на HTTP-эндпоинте `/metrics`; порт задаётся в конфигурации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

Разработка BunnyMQ сопровождается следующей программной документацией в соответствии с ГОСТ 19.201–78:

1. **Техническое задание** (ГОСТ 19.201–78) – настоящий документ; определяет требования к функциональным характеристикам, условиям эксплуатации, составу технических средств и порядку приёмки.
2. **Текст программы** (ГОСТ 19.401–78) – ссылка на репозиторий с исходным кодом брокера, клиентской библиотеки и утилиты командной строки.
3. **Пояснительная записка** (ГОСТ 19.404–79) – обоснование архитектурных и технических решений; описание алгоритмов хранения журнала, протокола Raft, работы FSM; выбор технологий; ожидаемые технико-экономические показатели.
4. **Руководство оператора** (ГОСТ 19.505–79) – инструкции по сборке, настройке конфигурационного файла, запуску кластера, работе с `bunnumq-cli`; описание параметров и диагностических сообщений.
5. **Программа и методика испытаний** (ГОСТ 19.301–79) – описание объекта и цели испытаний, применяемых средств тестирования, порядка проведения и методов проверки каждого требования ТЗ.

5.2. Специальные требования к документации

5.2.1. Требования к оформлению

- Все документы оформляются в соответствии с ГОСТ 19.106–78.
- Листы А4 с полями: левое не менее 30 мм, правое не менее 10 мм, верхнее и нижнее не менее 20 мм.
- Шрифт Times New Roman 12 pt, межстрочный интервал 1,5.
- Нумерация разделов, подразделов, таблиц и рисунков согласно ГОСТ 19.106–78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- Пояснительная записка загружается в систему проверки Антиплагиат ВШЭ не позднее чем за один рабочий день до защиты.

5.2.2. Требования к содержанию пояснительной записки

Пояснительная записка должна включать:

- описание предметной области и постановку задачи;
- обоснование выбора алгоритма консенсуса Raft и библиотеки dragonboat;
- описание структуры сегментированного журнала: Storage, SegmentStorage, LogSegment, OffsetIndexSegment, TimeIndexSegment;
- описание работы Partition FSM и Metadata FSM;
- обоснование выбора gRPC и Protobuf в качестве транспорта;
- описание алгоритма диапазонного назначения партиций в группах консьюмеров;
- описание механизма long-poll fetch через newDataCh;
- результаты бенчмарков записи на диск.

5.2.3. Требования к руководству оператора

Руководство оператора должно включать:

- описание конфигурационного файла и всех параметров;
- порядок запуска кластера из N узлов ($N \geq 3$);
- описание команд `bunnymq-cli`;
- примеры типовых операций через клиентскую библиотеку;
- описание метрик Prometheus и диагностических сообщений;
- порядок graceful shutdown узла.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Экономическая эффективность

Разработка выполняется в рамках учебного курсового проекта; коммерческая оценка стоимости не является целью ТЗ. Тем не менее использование BunnyMQ как решения с открытым исходным кодом имеет измеримые преимущества по сравнению с коммерческими альтернативами.

6.1.1. Снижение лицензионных затрат

Существующие промышленные брокеры сообщений с сопоставимыми гарантиями доставки (Apache Kafka в managed-варианте, Amazon SQS / Amazon MSK, Confluent Cloud) имеют стоимость эксплуатации от нескольких сотен до десятков тысяч долларов в месяц в зависимости от объёма трафика. BunnyMQ распространяется с открытым исходным кодом, что позволяет развернуть его на собственной инфраструктуре без лицензионных отчислений.

6.1.2. Снижение требований к инфраструктуре

Зависимости BunnyMQ ограничиваются одним исполняемым файлом на узел и дисковым хранилищем. Отсутствует необходимость в отдельном ZooKeeper-кластере (в отличие от Apache Kafka версий до 3.x), внешней базе данных или объектном хранилище. Это снижает операционную сложность и стоимость инфраструктуры.

6.1.3. Прогнозируемые технические характеристики

Таблица 6.1 — Ожидаемые технические характеристики BunnyMQ

Характеристика	Ожидаемое значение
Пропускная способность (acks=0, последовательная запись)	$\geq 100\,000$ сообщ./с на узел (размер 1 КБ)
Пропускная способность (acks=all, RF=3)	$\geq 10\,000$ сообщ./с на кластер
Задержка записи (p99, acks=all, RF=3, LAN)	≤ 50 мс
Задержка чтения (cached, long-poll timeout 0)	≤ 5 мс
Коэффициент готовности кластера (3 узла)	$\geq 0,999$ (переживает отказ одного узла)
Максимальный объём хранимых данных на узел	ограничен только ёмкостью диска

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Показатели производительности определены расчётным методом на основе бенчмарков последовательной записи на SSD (результаты в `benchmarks/disk_test.go`) и характеристик библиотеки dragonboat v4.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Стадии и этапы разработки определены в соответствии с ГОСТ 19.102–77.

Таблица 7.1 — Стадии и этапы разработки BunnyMQ

Стадия разработки	Этап работ	Содержание работ	Сроки
Техническое проектирование	Анализ требований	Составление ТЗ, изучение Apache Kafka, dragonboat v4, согласование с руководителем	До 15.01.26
	Проектирование архитектуры	Разработка дизайн-документов: Storage, Raft FSM, координаторы, API, клиентская библиотека, группы потребителей	До 15.01.26
Рабочее проектирование	Разработка Storage (M1)	LogSegment, OffsetIndexSegment, TimeIndexSegment, SegmentStorage, Storage, retention, recovery; unit-тесты	До 01.03.26
	Разработка Raft FSM (M2)	NodeHost wrapper, Metadata FSM, Partition FSM, snapshot; тесты детерминизма	До 15.03.26
	Разработка координаторов и API (M3)	ClusterCoordinator, DataCoordinator, gRPC-серверы, auth, клиентская библиотека Producer/Consumer/AdminClient, bunnymq-cli	До 01.04.26
	Разработка групп потребителей (M4)	GroupCoordinator, JoinGroup/Heartbeat/LeaveGroup, CommitOffset, Consumer group mode в pkg/client	До 10.04.26
Внедрение и испытание	Интеграция и тестирование	Сборка кластера из 3 узлов, сквозные тесты, нагрузочное тестирование	До 20.04.26
	Подготовка документации	Пояснительная записка, руководство оператора, программа и методика испытаний, текст программы	До 25.04.26
	Сдача проекта	Защита курсовой работы, демонстрация работающего кластера	

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7.1. Сроки разработки и исполнители

Разработка выполняется одним исполнителем: Б.А. Багавиев, группа БПИ233. Программный продукт (программа и документация) должен быть завершён не позднее **30.04.2026**.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

8.1. Критерии приёмки

BunnyMQ считается готовым к приёмке при выполнении всех следующих условий.

Качество кода:

- `go build ./...` завершается без ошибок;
- `go test ./...` проходит по всем пакетам;
- `golangci-lint run ./...` не выдаёт критических предупреждений;
- покрытие для `internal/storage` и `internal/metadata` не менее 80%.

Функциональность:

- все методы Data API и Management API из раздела 4 ТЗ реализованы и принимают запросы;
- кластер из 3 узлов создаёт топик, принимает сообщения с `acks=all` и отдаёт их через `Fetch`;
- при аварийной остановке одного узла из трёх кластер продолжает обслуживать запросы;
- Metadata FSM восстанавливается после перезапуска узла из снимка;
- Storage корректно выполняет восстановление после `torn write`.

Надёжность:

- сообщения, подтверждённые с `acks=all`, не теряются при отказе одного узла;
- очистка по политике хранения (`DeleteSegmentsBefore`) применяется детерминированно на всех репликах через команду `Raft`.

Производительность:

- пропускная способность последовательной записи на диск при размере блока 4 096 байт без `O_SYNC` не менее 500 МБ/с на тестовом стенде (подтверждается `benchmarks/disk_test.go`).

Группы консьюмеров:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- группа из двух консьюмеров корректно делит партиции через JoinGroup;
- при выходе одного консьюмера (LeaveGroup или session timeout) производится ребалансировка без потери данных;
- зафиксированные смещения сохраняются в Metadata FSM и доступны после перезапуска консьюмера.

8.2. Процедура приёмки

1. **Функциональное тестирование** – проверка каждого gRPC-метода через `bunnumq-cli` или автоматические тесты по сценариям из Программы и методики испытаний.
2. **Кластерное тестирование** – запуск трёх процессов `bunnumq` на локальной машине (разные порты); проверка `produce/fetch`, `failover` при остановке одного узла, восстановления после перезапуска.
3. **Модульное тестирование** – запуск `go test ./...`; проверка покрытия.
4. **Тестирование Storage** – корректность записи и чтения, сегментный переход, индексный поиск, восстановление после `torn write`.
5. **Бенчмарки** – запуск `go test ./benchmarks/... -bench=. -benchmem`; проверка соответствия характеристикам из таблицы 6.1.
6. **Проверка документации** – наличие всех документов, перечисленных в разделе 5 ТЗ.
7. **Составление акта приёмки** – по результатам составляется акт, подписываемый исполнителем и руководителем.

8.3. Требования к результатам приёмки

- `go test ./...` – статус PASS для всех пакетов.
- Отсутствие `data-race` при запуске `go test -race ./...`
- Покрытие `internal/storage` не менее 80% по отчёту `go tool cover`.
- Кластер из 3 узлов успешно завершает сквозной тест: `CreateTopic`, затем `Produce(acks=all) × 100`, затем `Fetch` всех сообщений и проверка смещений.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- Отсутствие потери подтверждённых сообщений после остановки одного узла и переизбрания лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ГЛОССАРИЙ И СОКРАЩЕНИЯ

Термины и сокращения, используемые в настоящем документе:

API — Application Programming Interface – программный интерфейс взаимодействия между компонентами системы.

Raft — Алгоритм консенсуса для распределённых систем, обеспечивающий согласованность реплицированного журнала при отказе меньшинства узлов.

dragonboat — Go-реализация алгоритма Raft (github.com/ltni/dragonboat/v4), используемая в BunnyMQ для репликации партиций и метаданных кластера.

gRPC — Google Remote Procedure Call – высокопроизводительный RPC-фреймворк на базе Protocol Buffers и HTTP/2, транспорт Data API и Management API.

Protobuf — Protocol Buffers – язык описания схем и механизм бинарной сериализации данных от Google.

Shard — Группа Raft-репликации (в терминологии dragonboat). В BunnyMQ один shard соответствует одной партиции; плюс один shard для метаданных кластера.

FSM — Finite State Machine – абстракция dragonboat для применения команд Raft к состоянию. В BunnyMQ: `IStateMachine (MetadataFSM)` и `IOndiskStateMachine (PartitionFSM)`.

Топик (Topic) — Именованный логический поток сообщений; разделяется на партиции для параллельной обработки. В тексте используется профессиональный термин «топик».

Партиция (Partition) — Независимый упорядоченный подпоток сообщений топика. Порядок гарантируется только внутри одной партиции.

Смещение (Offset) — Монотонно возрастающий 64-битный идентификатор позиции сообщения в партиции.

Продюсер (Producer) — Клиент, отправляющий сообщения в топики.

Консьюмер (Consumer) — Клиент, читающий сообщения из партиций.

Группа консьюмеров (Consumer Group) — Совокупность консьюмеров, совместно читающих топик; каждая партиция обрабатывается ровно одним членом группы.

acks=all — Режим подтверждения записи с ожиданием кворума реплик. Реализован через `dragonboat.SyncPropose`.

acks=0 — Публикация без ожидания подтверждения. Реализован через `dragonboat.Propose`.

Long-poll fetch — Режим чтения: запрос удерживается до появления новых данных или истечения `max_wait_ms`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Сегментированный журнал — Структура хранения: журнал разбит на файловые сегменты фиксированного максимального размера.

mmap — Memory-mapped file – отображение файла в адресное пространство процесса. Используется для индексных файлов (`.index`, `.timeindex`).

Ребалансировка — Перераспределение партиций между членами группы консьюмеров при изменении её состава.

NotLeader — Ответ gRPC: узел не является лидером нужной партиции; содержит адрес текущего лидера.

Prometheus — Система сбора метрик в формате time-series с pull-моделью.

zap — Go-библиотека структурированного логирования `go.uber.org/zap`.

ЕСПД — Единая система программной документации – комплекс стандартов ГОСТ 19.xxx.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

СРАВНЕНИЕ С АРАСНЕ KAFKA

BunnyMQ проектировался с опорой на архитектуру Apache Kafka с рядом упрощений.

Таблица 2.1 — Сравнение BunnyMQ и Apache Kafka

Характеристика	BunnyMQ	Apache Kafka
Консенсус / репликация	Raft (dragonboat v4), multi-raft	ISR + KRaft (с 3.x)
Формат данных на диске	Kafka-совместимый batch format	Kafka batch format
Транспорт клиента	gRPC + Protobuf	Kafka binary protocol (TCP)
acks=0	Поддерживается	Поддерживается
acks=1	Не реализован	Поддерживается
acks=all	Поддерживается	Поддерживается
Транзакции	Не поддерживаются	Поддерживаются (с 0.11)
Идемпотентный Producer	Не поддерживается	Поддерживается
Группы потребителей	Range assignment, v1	Range, RoundRobin, Sticky, Cooperative
Cooperative rebalance	Не поддерживается	Поддерживается (с 2.4)
Изменение состава кластера в runtime	Не поддерживается	Поддерживается
Хранение смещений групп	Metadata FSM (Raft)	Внутренний топик __consumer_offsets
Мониторинг	Prometheus text/plain	JMX + Prometheus exporter

Примечание. Формат batch_data на диске в BunnyMQ совпадает с форматом пакетов Kafka, что обеспечивает потенциальную совместимость на уровне хранилища. Транспортный протокол (gRPC вместо Kafka binary protocol) несовместим: существующие Kafka-клиенты не могут подключиться к BunnyMQ напрямую.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3

ПЕРЕЧЕНЬ gRPC-МЕТОДОВ

Все методы принадлежат пакету `bunnumq.v1`. Аутентификация — заголовок `bunnumq-auth-token` в gRPC-метаданных.

ManagementService

Таблица 3.1 — Методы ManagementService

Метод	Назначение	Тип
CreateTopic	Создание темы	Unary
DeleteTopic	Асинхронное удаление темы	Unary
ListTopics	Список тем кластера	Unary
DescribeTopic	Метаданные темы (лидеры, реплики)	Unary
AlterTopicPartitions	Увеличение числа партиций	Unary
AlterTopicRetention	Изменение параметров хранения	Unary
DescribeCluster	Состав кластера (узлы, адреса, статус)	Unary
ListPartitions	Партиции темы с метаданными	Unary

DataService

Таблица 3.2 — Методы DataService

Метод	Назначение	Тип
Produce	Запись пакета сообщений в партицию	Unary
Fetch	Чтение записей с long-poll	Unary
GetOffsets	Earliest / latest / by-timestamp смещения	Unary
JoinGroup	Регистрация потребителя в группе	Unary
Heartbeat	Подтверждение активности члена группы	Unary
LeaveGroup	Выход из группы	Unary
CommitOffset	Фиксация обработанных смещений	Unary
FetchCommittedOffsets	Чтение зафиксированных смещений группы	Unary

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Коды ошибок

Таблица 3.3 — Коды ошибок BunnyMQ

Код	Значение	gRPC status
OK	Успех	OK
InvalidArgument	Некорректные входные данные	INVALID_ARGUMENT
TopicNotFound	Тема не существует	NOT_FOUND
TopicAlreadyExists	Тема уже есть	ALREADY_EXISTS
PartitionNotFound	Партиция не существует	NOT_FOUND
NotLeader	Узел не лидер; ответ содержит адрес лидера	FAILED_PRECONDITION
OffsetOutOfRange	Смещение вне допустимого диапазона	OUT_OF_RANGE
MessageTooLarge	Размер пакета превышает допустимый	RESOURCE_EXHAUSTED
Unauthenticated	Токен отсутствует или недействителен	UNAUTHENTICATED
Unavailable	Нет лидера shard Raft	UNAVAILABLE
Timeout	Превышено время ожидания	DEADLINE_EXCEEDED

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4**СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ**

1. ГОСТ 19.101–77 Виды программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102–77 Стадии разработки // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103–77 Обозначения программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104–78 Основные надписи // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105–78 Общие требования к программным документам // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106–78 Требования к программным документам, выполненным печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201–78 Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301–79 Программа и методика испытаний // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401–78 Текст программы // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404–79 Пояснительная записка // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505–79 Руководство оператора // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603–78 Общие правила внесения изменений // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
13. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

14. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).
15. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
16. Apache Kafka Documentation. Версия 3.7. [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 26.04.2026).
17. gRPC Documentation. [Электронный ресурс]. — URL: <https://grpc.io/docs/> (дата обращения: 26.04.2026).
18. Protocol Buffers Documentation. Версия proto3. [Электронный ресурс]. — URL: <https://protobuf.dev/> (дата обращения: 26.04.2026).
19. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing. NetDB „11, 2011. [Электронный ресурс]. — URL: <https://notes.stephenholiday.com/Kafka.pdf> (дата обращения: 26.04.2026).
20. Uber go/zap. Blazing fast, structured, leveled logging in Go. [Электронный ресурс]. — URL: <https://github.com/uber-go/zap> (дата обращения: 26.04.2026).
21. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]